

EE 446: TinyML — Lab 6

TinyML for Anomaly Detection · Two-part lab (Edge Impulse + Quantized Autoencoder)

This report covers Part 1 (fan anomaly detection on Edge Impulse) and Part 2 (quantization-aware autoencoder for human-activity anomaly detection on Arduino Nano 33 BLE Sense).

Section 1 — Edge Impulse Results (Part 1)

Public Edge Impulse project link

<https://studio.edgeimpulse.com/public/1076612/latest>

Answers to the required Part 1 questions

1. Raw inputs	Three-axis accelerometer time series: accX, accY, accZ (100 Hz, 2000 ms window, 220 ms stride, zero-padded).
2. NN classifier features	Type: spectral features from the Spectral Analysis (DSP) block. Count: 33 input features. Example names: accX RMS, accX Peak 1 Height, accX Spectral Power 0.1–0.5, accY RMS, accZ Spectral Power 2.0–5.0 (also Peak/Freq and Spectral-Power bands per axis).
3. NN classifier outputs	2 classes: nominal, off. (The overall impulse output tile shows 3 = nominal, off, anomaly-score, because the K-means block appends an anomaly score.)
4. Anomaly detector features	Type: selected spectral features fed to a K-means detector. In the reference impulse the selected features are accX Peak 1 Height, accY Spectral Power 2.0–5.0, accZ Spectral Power 2.0–5.0 (3 features). Output: 1 anomaly score. <i>Confirm the exact selected set/count in your clone.</i>

Deployment evidence (screenshot)

```
nominal: 0.656250
off: 0.343750
Anomaly prediction: 56.150742
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 581 us, anomaly 571 us, postprocessing 388 us
#Classification predictions:
  nominal: 0.000000
  off: 0.996094
Anomaly prediction: 10.294332
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 523 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.597656
  off: 0.402344
Anomaly prediction: 757.701416
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 522 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.023437
  off: 0.976562
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 551 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.921875
  off: 0.078125
Anomaly prediction: 50.320477
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 529 us, postprocessing 78 us
#Classification predictions:
  nominal: 0.824219
  off: 0.175781
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 6 ms, anomaly 537 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.140625
  off: 0.859375
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 45 ms, inference 562 us, anomaly 549 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.562500
  off: 0.437500
Anomaly prediction: 15.089006
Starting inferencing in 2 seconds...
█
```

Live on-device inference on the Arduino Nano 33 BLE Sense (Edge Impulse run-impulse). Both classifier states are produced on hardware — strong **off** (e.g. 0.996) and strong **nominal** (e.g. 0.922) — with per-window timing (DSP 44 ms, classifier inference 6 ms, anomaly ≈ 0.55 ms) and K-means anomaly scores that rise sharply (up to 1000) on out-of-distribution motion.

Short interpretation

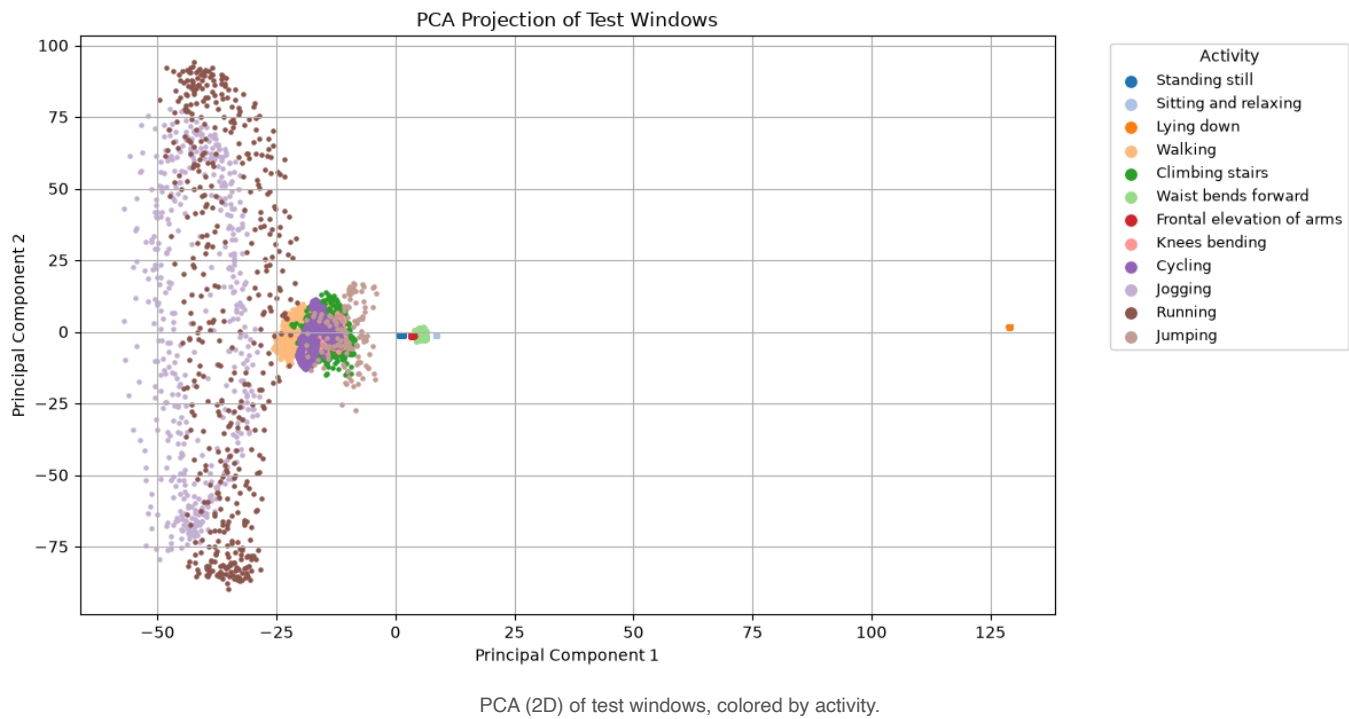
The reference classifier reports 100% validation accuracy and an ROC-AUC of 1.00. This looks perfect but should **not** be trusted unconditionally: the dataset is small and the windows overlap, so the validation split can leak information and inflate accuracy; performance on a different fan, mounting, or speed is likely lower. The classifier is reliable mainly when the input is **in-distribution** — which is exactly what the K-means **anomaly score** measures. In practice: trust the "nominal/off" label when the anomaly score is low (input resembles training data) and the operating conditions match those seen in training; treat the label as unreliable when the anomaly score is high (out-of-distribution input) or when the deployment hardware/environment differs from the training setup.

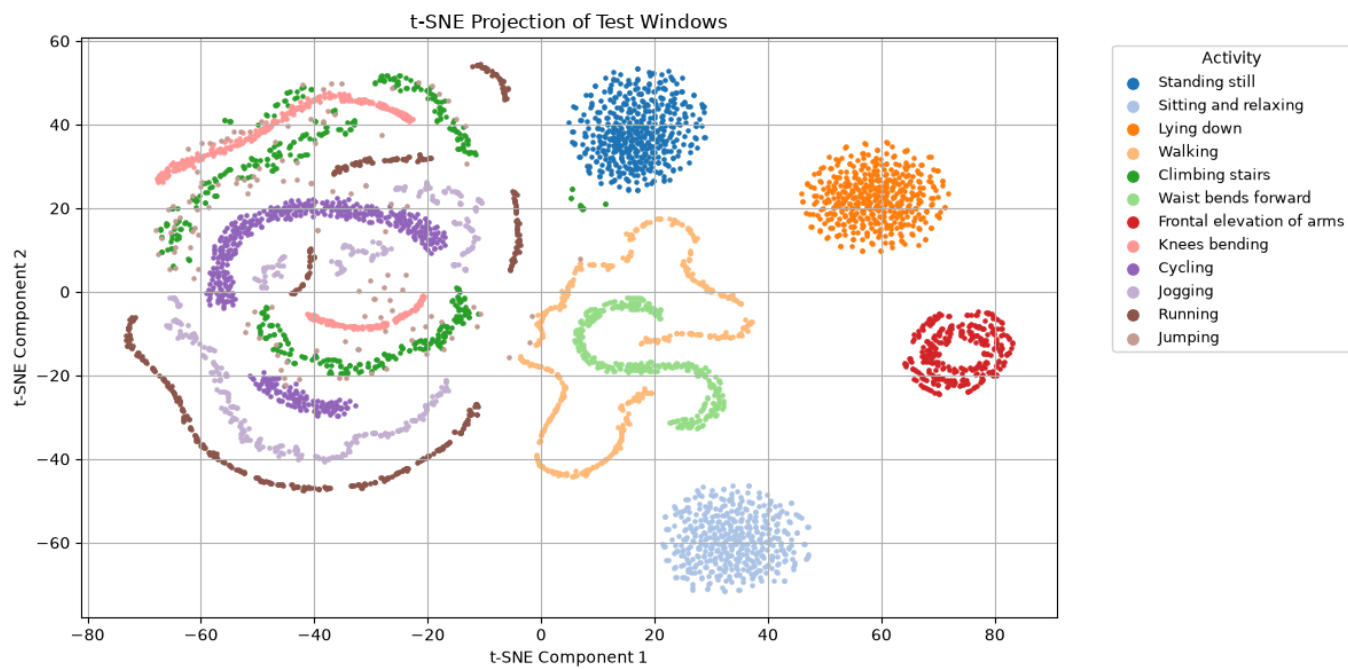
Section 2 — Autoencoder Model Results (Part 2)

Data preparation summary

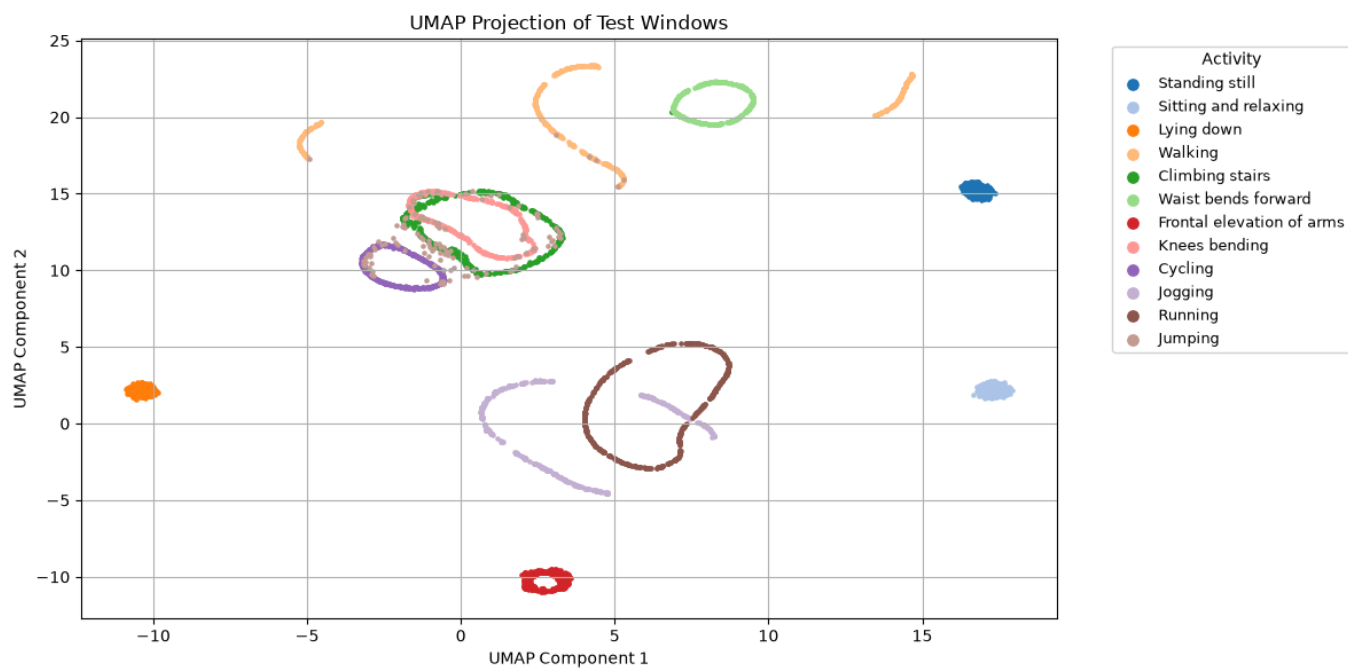
Input channels	Left-ankle accelerometer X/Y/Z (mHealth cols 6–8), label col 24
Windowing	100 samples × 3 axes → flattened to 300-dim vectors (stride 1), 70/30 temporal split per class
Normal group	standing, sitting, lying, waist bends, frontal arm elevation, knees bending (labels 0,1,2,5,6,7)
Train windows	21,350 (normal 10,478 / anomaly 10,872)
Test windows	8,479 (normal 4,155 / anomaly 4,324)

Dimensionality-reduction visualizations



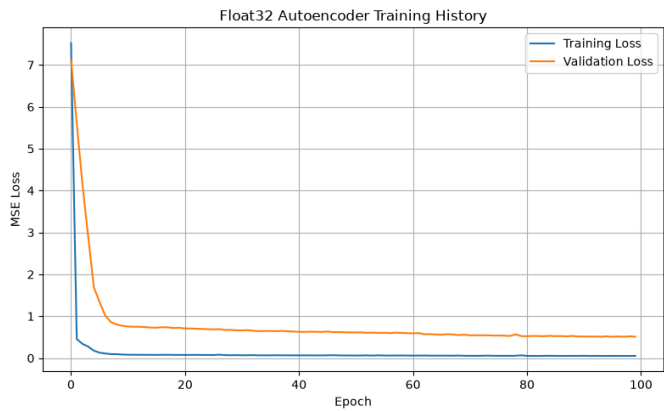


t-SNE (2D) of test windows.

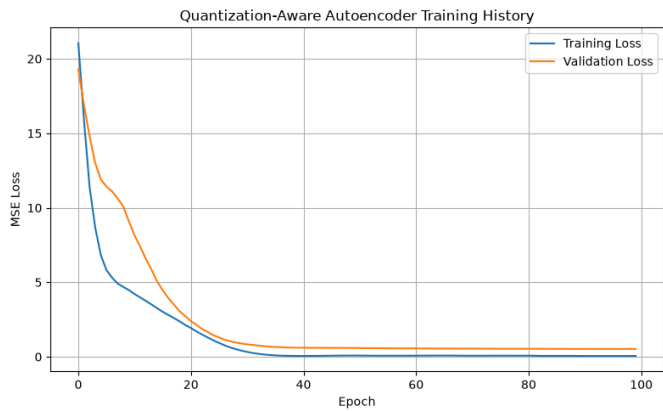


UMAP (2D) of test windows.

Training and validation loss curves

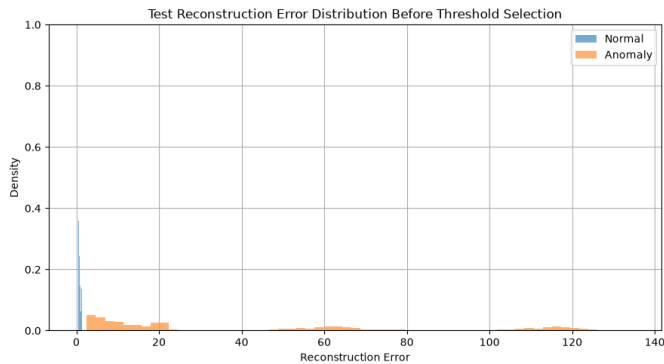


Float32 autoencoder training.

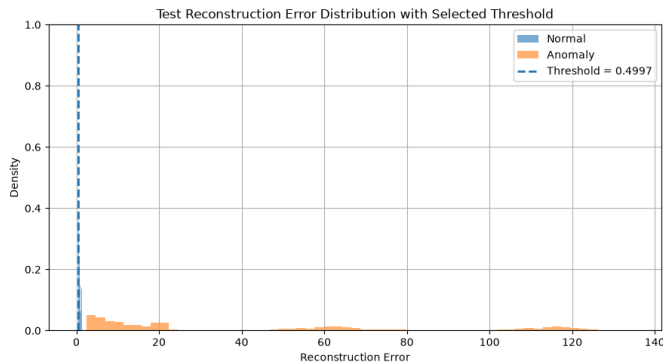


Quantization-aware training (QAT).

Reconstruction-error distribution & selected threshold



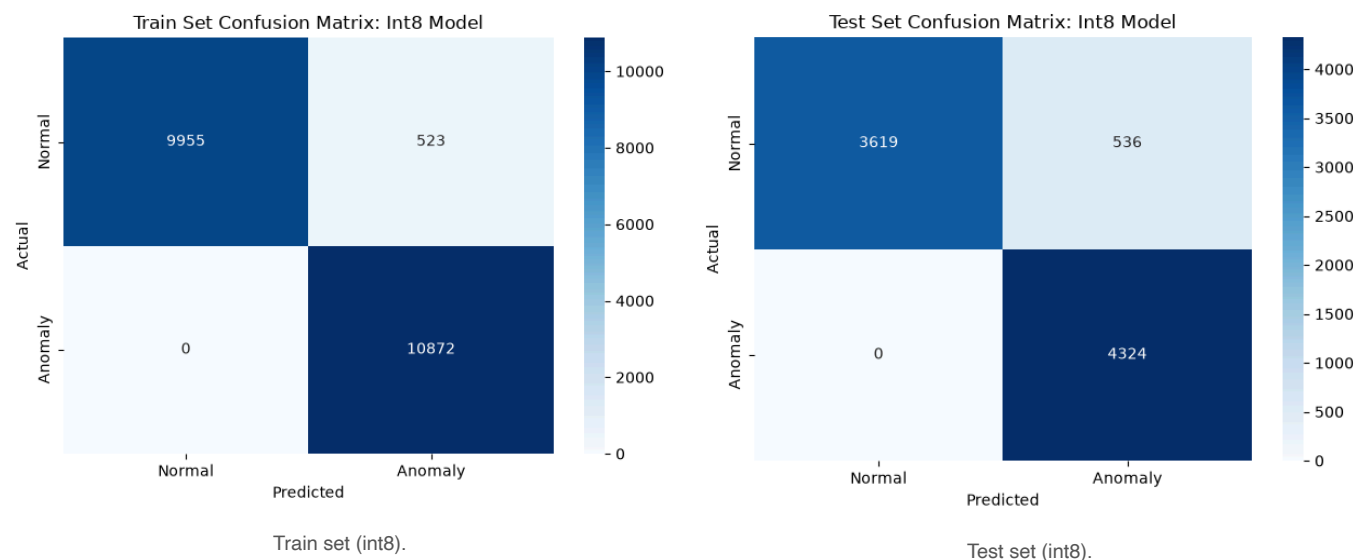
Before threshold selection.



With selected threshold.

Selected threshold	0.50 (MSE) — the 95th percentile of normal-training reconstruction error
Train-normal error	mean 0.105, max 1.105
Test-normal error	mean 0.145
Test-anomaly error	mean 43.0, min 2.53 (≈300× separation from normal)

Confusion matrices & evaluation metrics (int8 model)



Test set evaluation (int8 model)				
	precision	recall	f1-score	support
Normal (0)	1.00	0.87	0.93	4155
Anomaly (1)	0.89	1.00	0.94	4324
accuracy			0.94	8479
macro avg	0.94	0.94	0.94	8479
weighted avg	0.94	0.94	0.94	8479

Input quantization	scale 0.08136, zero-point 18
Output quantization	scale 0.08585, zero-point -8
Model size	Float32 TFLite 82.87 KB → Full int8 24.46 KB (g_model_len = 25,048 bytes)

Arduino serial monitor output

```
Reconstruction error: 0.533619
Result: Anomaly detected
Reconstruction error: 0.533269
Result: Anomaly detected
Reconstruction error: 0.473978
Result: Normal activity
Reconstruction error: 0.473941
Result: Normal activity
Reconstruction error: 0.535901
Result: Anomaly detected
Reconstruction error: 0.537045
Result: Anomaly detected
Reconstruction error: 0.537673
Result: Anomaly detected
Reconstruction error: 0.539468
Result: Anomaly detected
Reconstruction error: 0.496631
Result: Normal activity
Reconstruction error: 0.530508
Result: Anomaly detected
Reconstruction error: 0.534086
Result: Anomaly detected
Reconstruction error: 0.535462
Result: Anomaly detected
```

Autoencoder running on the Arduino Nano 33 BLE Sense (serial monitor, 115200 baud). With the board held near the ankle-normal orientation, reconstruction error hovers around the 0.50 threshold: windows below 0.50 (e.g. 0.474, 0.497) are classified **Normal activity**, while windows above (e.g. 0.531–0.539) are **Anomaly detected** — the decision boundary operating live on-device.

Deployment observation. The on-device reconstruction error is dominated by *sensor orientation*: flat on a table gives ≈ 90 , a hand-held vertical pose ≈ 2.5 – 5 , and only a precisely-held ankle-like orientation crosses below 0.50 into **Normal**. This directly illustrates why the deployed sensor must match the training sensor's mounting and units (see discussion below).

Section 3 — Discussion Questions

What threshold did you choose for anomaly detection, and why?

0.50 MSE, chosen as the 95th percentile of reconstruction error over *normal* training windows. Because the autoencoder is trained only on normal data, ~95% of normal windows fall below this value, giving a low false-positive rate, while the very large gap to anomaly errors (min ≈ 2.53 , mean ≈ 43) means every test anomaly is still caught (anomaly recall = 1.00). It is a fully unsupervised, data-driven choice that never uses anomaly labels.

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder learns a compressed representation that reconstructs the *normal* activity manifold well (low MSE ≈ 0.1). Anomalous, higher-motion activities lie outside that manifold, so the decoder reconstructs them poorly and the MSE jumps by $\sim 300\times$. Thresholding this single scalar separates the two classes without needing a labeled anomaly class.

What information from the Python notebook must be preserved when deploying to Arduino?

- The **int8 model bytes** (`autoencoder_model.cc / g_model`).
- The **input/output quantization parameters** (scale & zero-point) — the sketch reads these from the model at runtime, so they must travel with the model.
- The **window geometry**: 100 samples $\times$ 3 axes, row-major flatten order (`index = i*3 + j`).
- The **reconstruction-error threshold** (0.50).
- The **input definition & units**: left-ankle 3-axis accelerometer in m/s^2 , ~ 50 Hz sampling.

Why must the Arduino sketch use the same preprocessing assumptions as the notebook?

int8 quantization is calibrated to the training distribution. If the board feeds data with a different scale, axis order, or window length, the same physical motion maps to different int8 codes than the model expects, so reconstruction error — and therefore the 0.50 threshold — stops separating the classes. Concretely, the BMI270 returns acceleration in **g** while the mHealth data is in **m/s^2** , so the sketch multiplies each axis by **9.80665** to match training units; the flatten order and 100-sample window length must match as well.

What are the main limitations of this method on a microcontroller?

- **Fixed threshold, no adaptation** — sensor drift or a different wearer shifts the error distribution.
- **Quantization noise** — int8 rounding perturbs reconstructions; windows near the threshold get misclassified.
- **Weak generalization** — trained on one subject; the "normal vs anomaly" split is an arbitrary grouping of activities, not real fault detection.
- **Single sensor / fixed rate** — only left-ankle accel, 100-sample windows at ~ 50 Hz; orientation-sensitive.
- **No on-device learning** and limited memory (64 KB tensor arena, 24.5 KB model in flash); the detector flags novelty, not semantically meaningful anomalies.